RPOO — Corrigés commentés

Arbres AVL (ex. 18 à 21), listes doublement chaînées (ex. 30 à 33), ensembles chaînés (ex. 34 à 36) et ensembles ordonnés (ex. 37 à 40)

Ce que contient ce document

Pour chaque exercice : l'idée générale, le code Java complet, puis les points qui font perdre des points (cas limites, invariants, pièges de Java). Les exercices 22 à 29 ne figuraient pas dans les photos : ils ne sont pas traités ici.

Hypothèses sur les classes fournies

Je n'ai vu ni `ArbreBinaire`, ni `ChaineSimple`, ni `ChaineDouble`, ni les interfaces du package `col` en entier. J'ai donc fait les hypothèses suivantes ; si ta version diffère, seuls les noms ou l'ordre des paramètres changent.

Classe / interface	Hypothèse utilisée dans les corrigés
`ChaineSimple<E>`	champs `valeur` et `suivant` ; constructeur `ChaineSimple(E valeur, ChaineSimple<E> suivant)` (c'est celui que tu vois dans l'ex. 35 : `new ChaineSimple<>(null, debut)`)
`ChaineDouble<E>`	champs `valeur`, `precedent`, `suivant` ; constructeur `ChaineDouble(E valeur, ChaineDouble<E> precedent, ChaineDouble<E> suivant)` (**à adapter** à l'ordre réel de tes paramètres)
`ArbreBinaire<E extends Comparable<E>>`	champs `valeur`, `gauche`, `droit` ; constructeur `ArbreBinaire(E valeur)` ; méthodes `contient(E)`, `ajoute(E)` (renvoie un booléen), `hauteur()` (une feuille a pour hauteur 0) et `infixe()` (renvoie une `ChaineSimple<E>` triée par ordre croissant)
`Collection`, `List`, `Set`	signatures comme `java.util` : `E get(int)`, `E set(int, E)`, `void add(int, E)`, `E remove(int)`, `boolean add(E)`, `boolean remove(E)`, `boolean contains(E)`, `Object[] toArray()`, `boolean addAll(Collection<E>)`, `int indexOf(E)`
`SortedSet<E>`	`first()`, `last()` (exception `NoSuchElementException` si vide) et `SortedSet<E> subSet(E min, E max)` avec `min` inclus et `max` exclu

Vérification : tout le code ci-dessous a été compilé et exécuté sur des maquettes de ces classes, avec des tests aléatoires comparés à `TreeSet`, `ArrayList` et `HashSet` (arbre toujours AVL et trié, itérateurs, `equals`, `subSet`…). Il n'a pas été exécuté sur tes classes réelles : relance tes tests JUnit après avoir collé les méthodes.

1. Arbres AVL (exercices 18 à 21)

Rappels

Un arbre binaire de recherche (ABR) est d'autant plus efficace que sa hauteur est faible. Un arbre **AVL** est un ABR qui reste équilibré à chaque ajout et suppression.

- **Longueur de branche gauche** = hauteur du fils gauche + 1 (0 s'il n'y a pas de fils gauche). La longueur de branche droite est définie de la même façon.
- **Facteur d'équilibrage** (f.e.) = longueur de branche droite − longueur de branche gauche.
- L'arbre est AVL si, pour **chaque** sous-arbre, le f.e. vaut −1, 0 ou 1. Un f.e. de +2 (trop lourd à droite) ou −2 (trop lourd à gauche) déclenche une rotation.

Forme de l'arbre	f.e.
une feuille	0
seulement un fils gauche de hauteur h1	−h1 − 1
seulement un fils droit de hauteur h2	h2 + 1
deux fils (hauteurs h1 et h2)	h2 − h1

Exercice 18 — `supprime`

Idée. On supprime la valeur dans l'arbre en conservant la propriété d'ABR. Trois cas pour le nœud contenant la valeur : aucun fils (on le détache), un seul fils (le fils prend sa place), deux fils (on remplace sa valeur par le **maximum du sous-arbre gauche**, qui est son prédécesseur, puis on supprime ce maximum).

Pourquoi une exception sur une feuille ? La méthode s'applique à `this`, et un objet ne peut pas se « détruire lui-même » : si l'arbre est réduit à une racine sans fils, supprimer sa valeur donnerait un arbre vide, que `ArbreBinaire` ne peut pas représenter (ce serait `null`). C'est donc à la classe appelante de gérer ce cas (on le verra dans `SortedSetArbreBinaire.remove`, ex. 38).

Structure du code. Une méthode privée statique `retire(a, v)` renvoie la racine du sous-arbre `a` privé de `v` (éventuellement `null`). On l'utilise avec l'écriture `fils = retire(fils, v)`, qui gère proprement le cas où le fils disparaît. Seule la racine (`this`) est traitée à part, en recopiant le contenu du fils dans `this`.

```
public boolean supprime(E v) {
    if (gauche == null && droit == null)
        throw new IllegalStateException("arbre réduit à une feuille");
    if (!contient(v)) return false;

    int c = v.compareTo(valeur);
    if (c < 0) {
        gauche = retire(gauche, v);
    } else if (c > 0) {
        droit = retire(droit, v);
    } else if (gauche == null) {            // la racine n'a qu'un fils droit
        this.valeur = droit.valeur;         // on "remonte" le fils dans this
        this.gauche = droit.gauche;
        this.droit = droit.droit;           // this.droit est écrasé en dernier
    } else if (droit == null) {             // la racine n'a qu'un fils gauche
        this.valeur = gauche.valeur;
        this.droit = gauche.droit;
        this.gauche = gauche.gauche;
    } else {                                // deux fils : on remplace par le max du gauche
        E max = gauche.max();
        this.valeur = max;
        gauche = retire(gauche, max);
    }
    reequilibre();                          // ajout Ex. 21
    return true;
```

```
}

private E max() {
    return (droit == null) ? valeur : droit.max();
}

// renvoie la racine du sous-arbre a privé de v (null si le sous-arbre devient vide)
private static <T extends Comparable<T>> ArbreBinaire<T> retire(ArbreBinaire<T> a, T v) {
    if (a == null) return null;
    int c = v.compareTo(a.valeur);
    if (c < 0) {
        a.gauche = retire(a.gauche, v);
    } else if (c > 0) {
        a.droit = retire(a.droit, v);
    } else {
        if (a.gauche == null) return a.droit;
        if (a.droit == null) return a.gauche;
        T max = a.gauche.max();
        a.valeur = max;
        a.gauche = retire(a.gauche, max);
    }
    a.reequilibre();                        // ajout Ex. 21
    return a;
}
```

Les lignes marquées « ajout Ex. 21 » (`reequilibre()`) ne sont à écrire qu'à l'exercice 21. Pour l'exercice 18 seul, supprime-les.

Points d'attention

- Teste `contient(v)` d'abord : ainsi `supprime` renvoie `false` sans rien modifier si la valeur est absente.
- Dans le cas « un seul fils », `this.droit` (ou `this.gauche`) est écrasé **en dernier** : sinon on perdrait la référence au fils avant d'avoir copié ses champs.
- Le maximum du sous-arbre gauche est soit une feuille, soit un nœud n'ayant qu'un fils gauche : sa suppression par `retire` tombe donc toujours dans un cas simple.

Exercice 19 — `extraire`

Idée. Renvoyer, sous forme de chaîne simple triée, les valeurs de l'arbre comprises dans `[min, max[`. Un parcours infixe donne l'ordre croissant ; ici on le fait **à l'envers** (droit, racine, gauche) en ajoutant chaque valeur **en tête** d'une chaîne qu'on passe en paramètre. Les valeurs les plus grandes sont insérées d'abord, donc la chaîne finale est bien croissante, sans jamais avoir à parcourir la chaîne pour ajouter en fin.

```
public ChaineSimple<E> extraire(E min, E max) {
    return extraire(min, max, null);
}

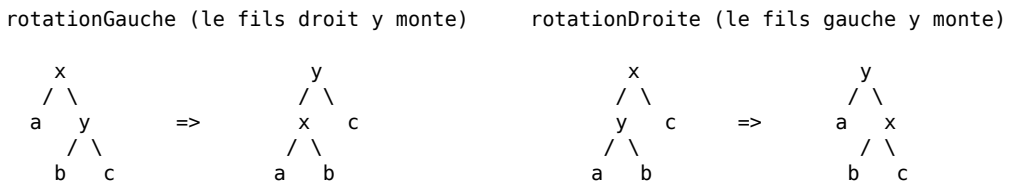
// ajoute en tête de "suite" les valeurs de [min, max[ de cet arbre, dans l'ordre croissant
private ChaineSimple<E> extraire(E min, E max, ChaineSimple<E> suite) {
    if (droit != null && valeur.compareTo(max) < 0)
        suite = droit.extraire(min, max, suite);
    if (valeur.compareTo(min) >= 0 && valeur.compareTo(max) < 0)
        suite = new ChaineSimple<>(valeur, suite);
    if (gauche != null && valeur.compareTo(min) > 0)
        suite = gauche.extraire(min, max, suite);
    return suite;
}
```

Élagage : ne pas visiter ce qui est inutile

- On visite le fils **droit** seulement si `valeur < max` (sinon tout le sous-arbre droit est ≥ `max`).
- On visite le fils **gauche** seulement si `valeur > min` (sinon tout le sous-arbre gauche est < `min`).
- Complexité : proportionnelle à la hauteur de l'arbre plus le nombre de valeurs renvoyées. Un résultat vide est représenté par `null`.

Exercice 20 — Rotations

Une rotation réorganise localement trois nœuds sans casser l'ordre de l'ABR. Notation : `a`, `b`, `c` sont des sous-arbres (éventuellement vides).



Astuce d'implémentation. Comme les méthodes s'appliquent à `this` et que la racine doit rester le même objet (sinon l'appelant garderait une référence vers un nœud qui n'est plus la racine), on **échange les valeurs** de `this` et du fils concerné, puis on réorganise les liens. Visuellement, `this` reçoit la valeur de `y` et le fils `y` reçoit l'ancienne valeur de `x` : l'arbre obtenu est exactement celui du schéma.

```
protected void rotationGauche() {
    if (droit == null) throw new IllegalStateException("pas de fils droit");
    ArbreBinaire<E> y = this.droit;
    E tmp = this.valeur; this.valeur = y.valeur; y.valeur = tmp;  // échange des valeurs
    this.droit = y.droit;      // c
    y.droit = y.gauche;        // b
    y.gauche = this.gauche;    // a
    this.gauche = y;           // y contient maintenant l'ancienne racine
}

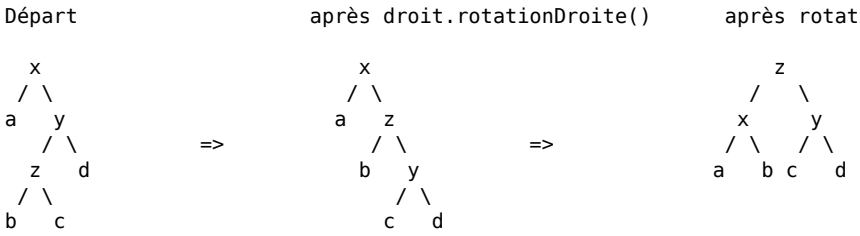
protected void rotationDroite() {
    if (gauche == null) throw new IllegalStateException("pas de fils gauche");
    ArbreBinaire<E> g = this.gauche;
    E tmp = this.valeur; this.valeur = g.valeur; g.valeur = tmp;
    this.gauche = g.gauche;    // a
    g.gauche = g.droit;        // b
    g.droit = this.droit;      // c
    this.droit = g;
}

protected void doubleRotationGauche() {   // cas droite-gauche
    droit.rotationDroite();
    rotationGauche();
}

protected void doubleRotationDroite() {   // cas gauche-droite
    gauche.rotationGauche();
    rotationDroite();
}
```

Rotations doubles

Une rotation simple ne suffit pas quand le déséquilibre est « en zigzag ». La rotation double est la composition de deux rotations simples. Exemple pour `doubleRotationGauche` (cas droite-gauche) :



Convention de nom : ici `doubleRotationGauche` traite le cas « fils droit plus lourd à gauche » (rotation droite du fils droit, puis rotation gauche de la racine). `doubleRotationDroite` est le cas symétrique. Si ton cours utilise la convention inverse, échange simplement les deux noms.

Exercice 21 — `facteurEquilibrage`, `reequilibre`, ajouts et suppressions AVL

```
protected int longueurBrancheGauche() {
    return (gauche == null) ? 0 : gauche.hauteur() + 1;
```

```
}

protected int longueurBrancheDroite() {
    return (droit == null) ? 0 : droit.hauteur() + 1;
}

public int facteurEquilibrage() {
    return longueurBrancheDroite() - longueurBrancheGauche();
}

protected void reequilibre() {
    int fe = facteurEquilibrage();
    if (fe > 1) {                                  // trop lourd à droite
        if (droit.facteurEquilibrage() < 0) doubleRotationGauche();
        else rotationGauche();
    } else if (fe < -1) {                          // trop lourd à gauche
        if (gauche.facteurEquilibrage() > 0) doubleRotationDroite();
        else rotationDroite();
    }
}
```

Quelle rotation choisir ? On regarde le f.e. de la racine, puis celui du fils du côté lourd :

f.e. de l'arbre	f.e. du fils lourd	Rotation à appliquer
+2 (trop lourd à droite)	≥ 0 (droit-droit)	`rotationGauche()`
+2 (trop lourd à droite)	< 0 (droit-gauche)	`doubleRotationGauche()`
−2 (trop lourd à gauche)	≤ 0 (gauche-gauche)	`rotationDroite()`
−2 (trop lourd à gauche)	> 0 (gauche-droite)	`doubleRotationDroite()`

Après un **ajout**, le f.e. du fils lourd n'est jamais nul. Après une **suppression**, il peut l'être : c'est pour cela que la condition est « ≥ 0 » (et non « > 0 ») pour la rotation simple.

Où appeler `reequilibre` ?

Dans `ajoute` et `supprime`, **à la remontée de la récursion** : après avoir ajouté (ou supprimé) dans un fils, chaque ancêtre se rééquilibre. Comme les rotations se font sur place, la racine reste le même objet et les appels récursifs n'ont rien à renvoyer. Version modifiée d'`ajoute` :

```
public boolean ajoute(E v) {
    int c = v.compareTo(valeur);
    if (c == 0) return false;
    boolean ok;
    if (c < 0) {
        if (gauche == null) { gauche = new ArbreBinaire<>(v); ok = true; }
        else ok = gauche.ajoute(v);
    } else {
        if (droit == null) { droit = new ArbreBinaire<>(v); ok = true; }
        else ok = droit.ajoute(v);
    }
    if (ok) reequilibre();      // ajout Ex. 21
    return ok;
}
```

Dans `supprime` et `retire` (ex. 18), l'appel `reequilibre()` figure juste avant le `return`, comme indiqué par les commentaires « ajout Ex. 21 ». Dans `retire`, il faut rééquilibrer **chaque nœud traversé** en remontant (`a.reequilibre()`), pas seulement la racine.

Performance : `hauteur()` est recalculée à chaque appel, donc `reequilibre` coûte O(taille du sous-arbre). C'est suffisant pour l'exercice. Une vraie implémentation stocke la hauteur dans chaque nœud pour rester en O(log n).

2. Liste doublement chaînée (exercices 30 à 33)

`ListeChaineDouble` mémorise `debut`, `fin`, et un **curseur** : `courant` (le dernier élément atteint par une méthode à indice) avec son indice `indiceCourant`, plus `size`. Ce curseur sert de cache pour accélérer les accès successifs.

Invariants à respecter dans toutes les méthodes

- Liste vide : `debut == fin == courant == null`, `indiceCourant == -1`, `size == 0`.
- Sinon, `courant` vaut soit `null` (avec `indiceCourant == -1`), soit le maillon d'indice `indiceCourant`.
- `debut.precedent == null` et `fin.suivant == null`.

Exercice 30 — `toString` et `positionne` (version simple)

`toString` : on part de `"("`, on ajoute `" " + valeur` pour chaque maillon, puis `" )"`. La liste vide donne `"( )"` et la liste 5, −8, 2 donne `"( 5 -8 2 )"`, conformément à l'énoncé.

`positionne(i)` (première version) : on parcourt depuis `debut` jusqu'à l'indice `i`, puis on met à jour `courant` et `indiceCourant`. La précondition (« `i` est valide ») n'est volontairement **pas** vérifiée ici : c'est aux appelants de le faire.

```
@Override
public String toString() {
    String s = "(";
    ChaineDouble<E> c = debut;
    while (c != null) {
        s += " " + c.valeur;
        c = c.suivant;
    }
    return s + " )";
}

protected void positionne(int i) {
    ChaineDouble<E> c = debut;
    for (int k = 0; k < i; k++) c = c.suivant;
    courant = c;
    indiceCourant = i;
}
```

Exercice 31 — Méthodes de `List`

L'ordre de l'énoncé compte : `add`, `get`, `set` et `remove` s'appuient sur `positionne`, les suivantes sur les précédentes. Chaque méthode vérifie d'abord l'indice, puis délègue à `positionne` « dans les cas utiles » : pas besoin de `positionne` pour ajouter en fin de liste.

```
private void verifieIndice(int i, int borneExclue) {
    if (i < 0 || i >= borneExclue)
        throw new IndexOutOfBoundsException("indice " + i + ", taille " + size);
}

@Override
public void add(int i, E e) {
    verifieIndice(i, size + 1);             // ici i = size est autorisé
    if (i == size) {                        // ajout en fin
        ChaineDouble<E> n = new ChaineDouble<>(e, fin, null);
        if (fin == null) debut = n; else fin.suivant = n;
        fin = n;
    } else {                                // insertion avant l'élément d'indice i
        positionne(i);
        ChaineDouble<E> p = courant.precedent;
        ChaineDouble<E> n = new ChaineDouble<>(e, p, courant);
        courant.precedent = n;
        if (p == null) debut = n; else p.suivant = n;
        courant = n;                        // n est maintenant l'élément d'indice i
    }
    size++;
}
```

```
@Override
public E get(int i) {
    verifieIndice(i, size);
    positionne(i);
    return courant.valeur;
}

@Override
public E set(int i, E e) {
    verifieIndice(i, size);
    positionne(i);
    E ancien = courant.valeur;
    courant.valeur = e;
    return ancien;
}

@Override
public E remove(int i) {
    verifieIndice(i, size);
    positionne(i);
    E ancien = courant.valeur;
    ChaineDouble<E> p = courant.precedent, s = courant.suivant;
    if (p == null) debut = s; else p.suivant = s;
    if (s == null) fin = p; else s.precedent = p;
    if (s != null) {
        courant = s;                        // l'élément suivant prend l'indice i
    } else {
        courant = p;                        // sinon on recule (null si liste vide)
        indiceCourant = i - 1;
    }
    size--;
    return ancien;
}

@Override
public int size() { return size; }

@Override
public void clear() {
    debut = null; fin = null; courant = null;
    indiceCourant = -1;
    size = 0;
}

@Override
public boolean addAll(Collection<E> c) {
    Object[] t = c.toArray();
    for (Object o : t) {
        @SuppressWarnings("unchecked") E e = (E) o;
        add(size, e);
    }
    return t.length > 0;
}

@Override
public int indexOf(E e) {
    int i = 0;
    for (ChaineDouble<E> c = debut; c != null; c = c.suivant, i++)
        if (Objects.equals(e, c.valeur)) return i;
    return -1;
}

@Override
public int lastIndexOf(E e) {
    int i = size - 1;
    for (ChaineDouble<E> c = fin; c != null; c = c.precedent, i--)
        if (Objects.equals(e, c.valeur)) return i;
    return -1;
}
```

Explications

- **Validité des indices** : pour `add`, `i` peut valoir `size` (ajout en fin) ; pour `get`, `set` et `remove`, `i` doit être strictement inférieur à `size`. D'où le paramètre `borneExclue` : `size + 1` pour `add`, `size` pour les autres.

- `add` **en fin** : on utilise `fin`, pas `positionne`. Cas particulier de la liste vide : le nouveau maillon devient aussi `debut`.
- `add` **au milieu / au début** : après `positionne(i)`, `courant` est l'élément qui sera poussé d'un cran. On insère le nouveau maillon avant lui, puis on fait de ce nouveau maillon le `courant` (il est bien à l'indice `i`, donc `indiceCourant` reste correct).
- `remove` : on relie le précédent et le suivant (en traitant les cas `debut` / `fin`). Ensuite on repositionne le curseur : sur le suivant s'il existe (il prend l'indice `i`), sinon sur le précédent (indice `i - 1`, ce qui donne `-1` et `null` quand la liste devient vide). Laisser `courant` sur le maillon supprimé briserait l'invariant.
- `addAll` : on s'appuie sur `add` en fin de liste. Passer par `toArray()` évite de dépendre d'un itérateur sur `Collection`.
- `indexOf` / `lastIndexOf` : parcours depuis `debut` (resp. `fin`) avec `Objects.equals`, qui gère aussi les valeurs `null`.

Exercice 32 — Méthodes de `Collection` et `equals`

On réutilise ce qui existe : `add(e)` = `add(size, e)`, `remove(e)` et `contains(e)` passent par `indexOf`.

```
@Override
public boolean add(E e) {
    add(size, e);
    return true;
}

@Override
public boolean remove(E e) {
    int i = indexOf(e);
    if (i < 0) return false;
    remove(i);
    return true;
}

@Override
public boolean contains(E e) {
    return indexOf(e) >= 0;
}

@Override
public Object[] toArray() {
    Object[] t = new Object[size];
    int k = 0;
    for (ChaineDouble<E> c = debut; c != null; c = c.suivant) t[k++] = c.valeur;
    return t;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof ListeChaineDouble)) return false;
    ListeChaineDouble<?> l = (ListeChaineDouble<?>) o;
    if (this.size != l.size) return false;
    ChaineDouble<E> a = this.debut;
    ChaineDouble<?> b = l.debut;
    while (a != null) {
        if (!Objects.equals(a.valeur, b.valeur)) return false;
        a = a.suivant;
        b = b.suivant;
    }
    return true;
}
```

Points d'attention

- **Surcharge** `remove(int)` / `remove(E)` : avec une `ListeChaineDouble<Integer>`, `l.remove(3)` appelle `remove(int)` (suppression à l'indice 3). Pour supprimer la **valeur** 3, écris `l.remove(Integer.valueOf(3))`.

- `equals` : deux listes sont égales si elles ont la même taille et les mêmes valeurs dans le même ordre. Tester d'abord `this == o`, puis le type, puis la taille évite des parcours inutiles. Si tu redéfinis `equals`, pense à redéfinir aussi `hashCode` de façon cohérente.

Exercice 33 — `positionne` optimisée

On a **trois points de départ** possibles : `debut` (indice 0), `fin` (indice `size - 1`) et `courant` (indice `indiceCourant`). On choisit celui dont la distance à `i` est la plus petite, puis on avance (avec `suivant`) ou on recule (avec `precedent`).

```
distance depuis debut   = i
distance depuis fin     = size - 1 - i
distance depuis courant = |i - indiceCourant|       (infinie si courant == null)

protected void positionne(int i) {
    int dDebut = i;                      // pas pour aller de debut à i
    int dFin = size - 1 - i;             // pas pour aller de fin à i
    int dCourant = (courant == null) ? Integer.MAX_VALUE : Math.abs(i - indiceCourant);

    if (dCourant <= dDebut && dCourant <= dFin) {        // départ : courant
        while (indiceCourant < i) { courant = courant.suivant;   indiceCourant++; }
        while (indiceCourant > i) { courant = courant.precedent; indiceCourant--; }
    } else if (dDebut <= dFin) {                          // départ : debut
        courant = debut; indiceCourant = 0;
        while (indiceCourant < i) { courant = courant.suivant; indiceCourant++; }
    } else {                                              // départ : fin
        courant = fin; indiceCourant = size - 1;
        while (indiceCourant > i) { courant = courant.precedent; indiceCourant--; }
    }
}
```

Points d'attention

- Si `courant == null` (liste fraîchement remplie par `add` en fin, par exemple), on ne peut pas partir de `courant` : on lui donne une distance infinie.
- Les trois branches laissent `courant` et `indiceCourant` cohérents : c'est la postcondition de l'énoncé.
- **Non-régression** : relance tous les tests des exercices 30 à 32. Ils doivent passer sans modification. Les erreurs typiques viennent d'un `indiceCourant` non mis à jour dans `add` ou `remove`.

3. Ensemble en chaîne simple (exercices 34 à 36)

`SetChaineSimple` mémorise `debut` (premier maillon) et `size`. L'ordre des éléments n'est pas significatif ; il n'y a ni `fin`, ni `precedent`.

Exercice 34 — `toString` et méthodes de `Collection`

Même schéma que pour la liste, avec le format `"{ 5 -8 2 }"` (ensemble vide : `"{ }"`). Pour `add`, l'énoncé impose de refuser les doublons : on teste `contains` d'abord. L'ordre n'important pas, on insère **en tête** de chaîne, en O(1).

```
@Override
public String toString() {
    String s = "{";
    for (ChaineSimple<E> c = debut; c != null; c = c.suivant)
        s += " " + c.valeur;
    return s + " }";
}

@Override
public int size() { return size; }

@Override
public void clear() {
    debut = null;
    size = 0;
}

@Override
public boolean contains(E e) {
    for (ChaineSimple<E> c = debut; c != null; c = c.suivant)
        if (Objects.equals(e, c.valeur)) return true;
    return false;
}

@Override
public boolean add(E e) {
    if (contains(e)) return false;
    debut = new ChaineSimple<>(e, debut);    // insertion en tête : O(1)
    size++;
    return true;
}

@Override
public boolean remove(E e) {
    ChaineSimple<E> prec = null, c = debut;
    while (c != null && !Objects.equals(e, c.valeur)) {
        prec = c;
        c = c.suivant;
    }
    if (c == null) return false;             // absent
    if (prec == null) debut = c.suivant;     // suppression du premier maillon
    else prec.suivant = c.suivant;
    size--;
    return true;
}

@Override
public Object[] toArray() {
    Object[] t = new Object[size];
    int k = 0;
    for (ChaineSimple<E> c = debut; c != null; c = c.suivant) t[k++] = c.valeur;
    return t;
}
```

Points d'attention

- Dans `remove`, on garde le maillon **précédent** (`prec`) pendant le parcours, car un maillon simple ne connaît pas son précédent. Si `prec == null`, on supprime le premier maillon et c'est `debut` qui change.

- N'oublie pas de décrémenter `size` à chaque suppression réussie, et de le remettre à 0 dans `clear`.

Exercice 35 — L'itérateur `SetChaineSimpleIterator`

L'itérateur est une **classe interne** : elle accède directement aux attributs de l'ensemble (`SetChaineSimple.this.debut`). Il maintient deux références :

- `prevNext` : le maillon **qui précède** le prochain élément à lire (donc `prevNext.suivant` est cet élément) ;
- `prevRemove` : le maillon qui précède le dernier élément renvoyé par `next` (donc `prevRemove.suivant` est l'élément à supprimer par `remove`), ou `null` si `next` n'a pas été appelé depuis le dernier `remove`.

Pourquoi des maillons « précédents » ? Parce qu'on ne peut pas retirer un maillon d'une chaîne simple sans modifier son précédent. Pour que le premier élément ait lui aussi un précédent, le constructeur crée un **maillon fictif** (sentinelle) dont `suivant` pointe vers `debut`.

```
@Override
public Iterator<E> iterator() { return new SetChaineSimpleIterator(); }

private class SetChaineSimpleIterator implements Iterator<E> {
    private ChaineSimple<E> prevNext;     // prevNext.suivant = prochain élément à lire
    private ChaineSimple<E> prevRemove;   // prevRemove.suivant = élément à supprimer

    public SetChaineSimpleIterator() {    // maillon fictif devant le premier
        this.prevNext = new ChaineSimple<>(null, SetChaineSimple.this.debut);
        this.prevRemove = null;
    }

    @Override
    public boolean hasNext() {
        return prevNext.suivant != null;
    }

    @Override
    public E next() {
        if (!hasNext()) throw new IllegalStateException();
        prevRemove = prevNext;
        prevNext = prevNext.suivant;
        return prevNext.valeur;
    }

    @Override
    public void remove() {
        if (prevRemove == null) throw new IllegalStateException();
        ChaineSimple<E> cible = prevRemove.suivant;
        if (cible == SetChaineSimple.this.debut)
            SetChaineSimple.this.debut = cible.suivant;   // on supprime le premier
        prevRemove.suivant = cible.suivant;               // saute le maillon
        prevNext = prevRemove;                            // le prochain à lire suit prevRemove
        prevRemove = null;                                // pas deux remove d'affilée
        SetChaineSimple.this.size--;
    }
}
```

Déroulé sur la chaîne a → b → c

```
Départ        : prevNext = [sentinelle] -> a -> b -> c        prevRemove = null
next()  -> a  : prevRemove = sentinelle ; prevNext = a
next()  -> b  : prevRemove = a          ; prevNext = b
remove()      : a.suivant = c (b est sauté) ; prevNext = a ; prevRemove = null
next()  -> c  : prevRemove = a          ; prevNext = c
```

Points d'attention

- Dans `remove`, si l'élément à supprimer est le premier (`cible == debut`), il faut mettre à jour `debut` de l'ensemble, en plus de la sentinelle : c'est le « éventuellement `debut` » de l'énoncé. Pense aussi à `size--`.

- Après `remove`, on fait `prevNext = prevRemove` : sans cela, `prevNext` resterait sur le maillon supprimé et le `next` suivant relierait le mauvais maillon.
- `prevRemove = null` après `remove` : deux `remove` d'affilée, ou un `remove` avant tout `next`, lèvent donc `IllegalStateException`, comme demandé par l'énoncé. Même exception pour `next` quand `hasNext()` est faux.

Exercice 36 — Méthodes ensemblistes par défaut (interface `Set`)

Ces quatre méthodes sont écrites une fois pour toutes dans l'interface, uniquement avec `contains`, `add` et des itérateurs (donc valables pour n'importe quelle implémentation). Dans l'illustration de l'énoncé, le récepteur `s1` est modifié ; le résultat est `true` si `s1` a changé.

```
default boolean containsAll(Set<E> c) {
    Iterator<E> it = c.iterator();
    while (it.hasNext()) {
        if (!this.contains(it.next())) return false;
    }
    return true;
}

default boolean addAll(Set<E> c) {
    boolean modifie = false;
    Iterator<E> it = c.iterator();
    while (it.hasNext()) {
        if (this.add(it.next())) modifie = true;
    }
    return modifie;
}

default boolean retainAll(Set<E> c) {
    boolean modifie = false;
    Iterator<E> it = this.iterator();
    while (it.hasNext()) {
        if (!c.contains(it.next())) {
            it.remove();
            modifie = true;
        }
    }
    return modifie;
}

default boolean removeAll(Set<E> c) {
    boolean modifie = false;
    Iterator<E> it = this.iterator();
    while (it.hasNext()) {
        if (c.contains(it.next())) {
            it.remove();
            modifie = true;
        }
    }
    return modifie;
}
```

Explications et pièges

- `containsAll` : tous les éléments de `c` doivent appartenir au récepteur. On s'arrête au premier absent.
- `addAll` **(union)** : `modifie = true` dès qu'un `add` réussit. N'écris pas `modifie = modifie || this.add(...)` : dès que `modifie` vaut `true`, l'opérateur `||` n'évalue plus `add` et les éléments suivants ne seraient plus ajoutés.
- `retainAll` **(intersection) et** `removeAll` **(différence)** : on parcourt le **récepteur** et on supprime avec `it.remove()`. Appeler `this.remove(...)` pendant le parcours casserait l'itérateur.

`equals` dans `SetChaineSimple`

Deux ensembles sont égaux si et seulement s'ils contiennent exactement les mêmes éléments. Comme il n'y a pas de doublons, il suffit de comparer les tailles puis de vérifier l'inclusion avec

`containsAll`.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Set)) return false;
    @SuppressWarnings("unchecked")
    Set<E> s = (Set<E>) o;
    return this.size() == s.size() && this.containsAll(s);
}
```

4. Ensembles ordonnés (exercices 37 à 40)

Exercice 37 — `SortedSetChaineSimple` (mise en œuvre dérivée)

La classe **hérite** de `SetChaineSimple` : `add`, `remove`, `contains`, `iterator`, etc. sont déjà là. Il reste à écrire les méthodes de `SortedSet`. La chaîne héritée n'est pas triée, donc chaque méthode parcourt toute la chaîne.

```
@Override
public E first() {
    if (debut == null) throw new NoSuchElementException();
    E min = debut.valeur;
    for (ChaineSimple<E> c = debut.suivant; c != null; c = c.suivant)
        if (c.valeur.compareTo(min) < 0) min = c.valeur;
    return min;
}

@Override
public E last() {
    if (debut == null) throw new NoSuchElementException();
    E max = debut.valeur;
    for (ChaineSimple<E> c = debut.suivant; c != null; c = c.suivant)
        if (c.valeur.compareTo(max) > 0) max = c.valeur;
    return max;
}

@Override
public SortedSet<E> subSet(E min, E max) {
    SortedSetChaineSimple<E> res = new SortedSetChaineSimple<>();
    for (ChaineSimple<E> c = debut; c != null; c = c.suivant)
        if (c.valeur.compareTo(min) >= 0 && c.valeur.compareTo(max) < 0)
            res.add(c.valeur);
    return res;
}
```

- `first` et `last` : un parcours qui retient le plus petit (resp. le plus grand) élément avec `compareTo`. Ensemble vide : `NoSuchElementException`, comme l'indique la documentation de l'interface.
- `subSet(min, max)` : on construit un **nouvel** ensemble avec les éléments `e` tels que `min ≤ e < max`.
- Coût : O(n) pour chaque méthode. C'est le défaut de cette version, que l'arbre AVL améliore.

Exercice 38 — `SortedSetArbreBinaire` : `toString`, `equals`, méthodes de `Collection`

Ici, l'ensemble est représenté par un attribut `arbre` (un `ArbreBinaire`, ou `null` si l'ensemble est vide) et par `size`. L'arbre ne représentant pas l'ensemble vide, `null` joue ce rôle.

```
@Override
public String toString() {
    String s = "{";
    if (arbre != null)
        for (ChaineSimple<E> c = arbre.infixe(); c != null; c = c.suivant)
            s += " " + c.valeur;
    return s + " }";
}

@Override
public boolean equals(Object o) {          // recopiée de SetChaineSimple
    if (this == o) return true;
    if (!(o instanceof Set)) return false;
    @SuppressWarnings("unchecked")
    Set<E> s = (Set<E>) o;
    return this.size() == s.size() && this.containsAll(s);
}

@Override
public int size() { return size; }

@Override
public void clear() {
    arbre = null;
    size = 0;
```

```
}

@Override
public boolean contains(E e) {
    return arbre != null && arbre.contient(e);
}

@Override
public boolean add(E e) {
    if (arbre == null) {                    // ensemble vide : on crée la racine
        arbre = new ArbreBinaire<>(e);
        size = 1;
        return true;
    }
    boolean ajoute = arbre.ajoute(e);
    if (ajoute) size++;
    return ajoute;
}

@Override
public boolean remove(E e) {
    if (arbre == null) return false;
    if (size == 1) {                        // l'arbre est une feuille : supprime() lèverait une exception
        if (!arbre.contient(e)) return false;
        arbre = null;
        size = 0;
        return true;
    }
    boolean supprime = arbre.supprime(e);
    if (supprime) size--;
    return supprime;
}

@Override
public Object[] toArray() {
    Object[] t = new Object[size];
    if (arbre != null) {
        int k = 0;
        for (ChaineSimple<E> c = arbre.infixe(); c != null; c = c.suivant)
            t[k++] = c.valeur;
    }
    return t;
}
```

Explications

- `toString` : même format que `SetChaineSimple`, mais on parcourt `arbre.infixe()`, donc les éléments sortent triés.
- `equals` : recopiée de `SetChaineSimple`, sans adaptation. Elle utilise `containsAll`, donc `iterator` : ses tests ne peuvent passer qu'après l'exercice 39, comme le prévient l'énoncé.
- `add` : si l'arbre est `null`, on crée la racine ; sinon on délègue à `arbre.ajoute(e)` et on incrémente `size` seulement si l'ajout a eu lieu.
- `remove` : c'est le piège de l'exercice. `ArbreBinaire.supprime` lève `IllegalStateException` si l'arbre est une feuille. Or `size == 1` signifie exactement que l'arbre est une feuille : on gère alors ce cas à la main (`arbre = null` si l'élément est bien là).

Exercice 39 — L'itérateur `SortedSetArbreBinaireIterator`

Au lieu de parcourir l'arbre directement, l'itérateur travaille sur un **instantané trié** : la chaîne `arbre.infixe()` calculée à sa création. `valeurs` est la partie de la chaîne qu'il reste à lire, et `remove` le maillon du dernier élément renvoyé par `next`.

```
@Override
public Iterator<E> iterator() { return new SortedSetArbreBinaireIterator(); }

private class SortedSetArbreBinaireIterator implements Iterator<E> {
    private ChaineSimple<E> valeurs;   // valeurs restant à lire (ordre croissant)
    private ChaineSimple<E> remove;    // dernier maillon renvoyé par next
```

```
    public SortedSetArbreBinaireIterator() {
        if (SortedSetArbreBinaire.this.arbre == null)
            this.valeurs = null;
        else this.valeurs = SortedSetArbreBinaire.this.arbre.infixe();
        this.remove = null;
    }

    @Override
    public boolean hasNext() {
        return valeurs != null;
    }

    @Override
    public E next() {
        if (valeurs == null) throw new IllegalStateException();
        remove = valeurs;
        valeurs = valeurs.suivant;
        return remove.valeur;
    }

    @Override
    public void remove() {
        if (remove == null) throw new IllegalStateException();
        SortedSetArbreBinaire.this.remove(remove.valeur);   // supprime dans l'arbre
        remove = null;
    }
}
```

Points d'attention

- **Appel qualifié** : dans la classe interne, le nom `remove` désigne l'`remove()` de l'itérateur (sans paramètre). Java ne cherche alors plus dans la classe englobante : on doit écrire `SortedSetArbreBinaire.this.remove(...)` pour appeler `remove(E)` de l'ensemble. Le champ `remove` (un maillon) et la méthode `remove()` ne se gênent pas.
- Supprimer dans l'arbre ne perturbe pas le parcours, puisque `valeurs` est une copie indépendante de l'arbre.
- `remove` est remis à `null` après chaque suppression : deux `remove` consécutifs lèvent `IllegalStateException`.
- Constructeur : si l'arbre est `null` (ensemble vide), `valeurs` vaut `null` et `hasNext()` renvoie `false`.

Exercice 40 — `first`, `last`, `subSet` dans `SortedSetArbreBinaire`

L'exercice 19 a été écrit pour ça : `extraire(min, max)` renvoie déjà la chaîne triée des éléments de `[min, max[`. Il reste à les mettre dans un nouvel ensemble. Pour `first` et `last`, on exploite le fait que `infixe()` est trié : le premier maillon est le minimum, le dernier est le maximum.

```
@Override
public E first() {
    if (arbre == null) throw new NoSuchElementException();
    return arbre.infixe().valeur;
}

@Override
public E last() {
    if (arbre == null) throw new NoSuchElementException();
    ChaineSimple<E> c = arbre.infixe();
    while (c.suivant != null) c = c.suivant;
    return c.valeur;
}

@Override
public SortedSet<E> subSet(E min, E max) {
    SortedSetArbreBinaire<E> res = new SortedSetArbreBinaire<>();
    if (arbre != null)
        for (ChaineSimple<E> c = arbre.extraire(min, max); c != null; c = c.suivant)
            res.add(c.valeur);
    return res;
}
```

- Ensemble vide : `NoSuchElementException` pour `first` et `last`, comme dans la version dérivée.
- Les valeurs de `extraire` arrivent **dans l'ordre croissant** ; grâce au rééquilibrage de l'exercice 21, le nouvel arbre construit par `add` reste équilibré (sans AVL, on obtiendrait une liste dégénérée).
- Si ton `ArbreBinaire` expose ses fils, `first` et `last` peuvent s'écrire en O(hauteur) en descendant toujours à gauche (resp. à droite). La version ci-dessus est en O(n) mais ne dépend d'aucun accesseur.

5. À retenir et check-list avant de rendre

Pièges classiques de ces exercices

- Mettre à jour **tous** les attributs de structure : `size`, `debut`, `fin`, `courant`, `indiceCourant`.
- Traiter les cas limites : liste ou ensemble vide, un seul élément, premier ou dernier maillon.
- Valider l'indice **avant** d'appeler `positionne`, et ne jamais vérifier la précondition dans `positionne` elle-même.
- Dans un itérateur : supprimer via `it.remove()`, jamais via la collection pendant le parcours.
- Comparer avec `Objects.equals` (valeurs `null`) et avec `compareTo` pour les types ordonnés.
- Rotations AVL : échanger les valeurs pour garder la même racine, et appeler `reequilibre` à la remontée de la récursion.

Ordre de travail conseillé

- Écrire une méthode, lancer ses tests, passer à la suivante (l'énoncé impose l'ordre de la plupart des méthodes).
- Après l'exercice 33, relancer **tous** les tests des exercices 30 à 32 (non-régression).
- Après l'exercice 21, relancer les tests de `ajoute` et `supprime` : l'arbre doit rester trié **et** équilibré.